



B

WINDOWS FUNCTIONS USED FOR EVASION



This appendix describes Windows functions that are commonly used for some of the evasion techniques discussed in this book. While it's not a comprehensive list of functions that might be abused by threat actors, these are some of the ones I believe are the most interesting or important to be familiar with for the purposes of malware analysis.

Note that the functions are written without their *A* and *W* suffixes. For example, `CreateFileW` is listed only as `CreateFile`. Also, some functions have both an *Nt* and a *Zw* variant, such as `NtLoadDriver` and `ZwLoadDriver`, but only the *Nt* variant is listed here. For more information on these variations, see [Chapter 1](#).

You can find a more complete list of commonly abused functions at <https://malapi.io>.

AddAtom

Adds a string to the local atom table. Can be used as part of certain process injection techniques, such as atom bombing.

AddVectoredExceptionHandler

Registers a new vectored exception handler. Can be used to abuse exceptions for the purposes of anti-analysis and anti-debugging. An alternative function is `RtlAddVectoredExceptionHandler`.

AdjustTokenPrivileges

Enables or disables privileges of an access token. Can be abused to elevate privileges.

BCrypt*

See the `Crypt*` functions. All functions that begin with `BCrypt*` (such as `BCryptEncrypt` and `BCryptDestroyKey`) align to their equivalent `Crypt*` function (such as `CryptEncrypt` and `CryptDestroyKey`).

BlockInput

Blocks input from reaching applications. Can be used as an anti-debugging technique to block interaction with the debugger.

CallNtPowerInformation

Returns information such as the battery state and last sleep time, which can be used to infer if the system is a VM.

CheckRemoteDebuggerPresent

Returns a nonzero value if the current process is being debugged; otherwise, returns zero. Can be used to detect a debugger in use.

CloseHandle

Closes an open handle. Can be used as an anti-debugging technique to crash certain debuggers under specific circumstances. An alternative is the now-deprecated `NtClose` function.

CreateFile

Gets a handle to a file or creates a new file. Can be used for many purposes, including VM and sandbox detection (to search for hypervisor-related files and pipes, for example). An alternative is the `NtCreateFile` function.

CreateFileTransacted

Creates or opens a file as an NTFS transacted operation. Can be used for process manipulation, such as process doppelganging.

CreateMutex(Ex)

Opens a mutex object or creates a new mutex. Can be used to enumerate mutexes related to hypervisors as a VM and sandbox detection technique.

CreateProcess

Creates a new process and is often called as part of process injection or during unpacking. Alternatives are the `NtCreateProcess(Ex)`, `CreateProcessInternal`, and `NtCreateUserProcess` functions.

CreateProcessWithToken

Creates a new process, assigning it the permissions of an existing token. Can be abused for privilege elevation and defense evasion.

CreateRemoteThread

Executes a new thread in the address space of another process and is often used as part of various process injection techniques. Alternatives include the `NtCreateThreadEx` and `RtlCreateUserThread` functions.

CreateService

Creates a service. Can be used for persistence or for loading malicious kernel modules.

CreateToolhelp32Snapshot

Creates a snapshot of the currently running processes on the system. Can be used to locate processes on the system for injection or for VM detection. Typically called before `GetProcess32First` and `GetProcess32Next`.

CreateTransaction

Creates a new NTFS transaction object. An alternative is the `NtCreateTransaction` function. *See also* `CreateFileTransacted`.

Crypt*

Functions that begin with `Crypt` (such as `CryptEncrypt`, `CryptDecrypt`, and `CryptCreateHash`) are used for various cryptographic operations, such as encrypting and hashing data, often for the purposes of obfuscation and defense evasion.

DebugActiveProcess

Enables a debugger to attach to an active process. Can be used to detect a debugger.

DeleteFile

Deletes a file or directory. Can be used to hide artifacts on disk by removing them. An alternative is the `NtDeleteFile` function.

DeviceIOControl

Allows a process in user space to send a control code to a driver in kernel space. Often used by rootkits to send control codes to malicious kernel drivers.

DsGetDcName

Retrieves the name of the system's domain controller and is often used to detect if the system is part of a domain for context awareness, targeting, or sandbox detection purposes.

DuplicateToken(Ex)

Creates a “copy” of a token assigned to another process. Often followed by a call to a function such as `ImpersonateLoggedOnUser` as part of privilege elevation and defense evasion.

EnumDisplayMonitors

Identifies the number of monitors configured on the system to infer if the system is a VM or sandbox.

EnumServiceStatus(Ex)

Enumerates services on the system. Can be used to identify hypervisor-related services for VM detection.

EnumSystemFirmwareTables

Enumerates system firmware tables. Can be used to identify hardware on the system and detect a VM.

EnumWindows

Enumerates open windows. Can be used to detect malware analysis tools and debuggers.

ExitProcess

Ends a process and can be used as an anti-analysis technique.

ExitWindows(Ex)

Logs out of the current user account or shuts down the system. Can be used as an anti-analysis and anti-sandbox technique.

FindFirstFile(Ex)

Enumerates files on the filesystem. Can be used to locate hypervisor-related files for VM and sandbox detection. Called prior to `FindNextFile`, which iterates through each file on the system.

FindFirstUrlCacheEntry(Ex)

Enumerates the browser cache. Lack of browser cache data could indicate a VM or sandbox environment. Called prior to `FindNextUrlCacheEntry(Ex)`.

FindWindow(Ex)

Locates a certain window, such as a specific analysis tool or debugger.

FltEnumerateFilters

Can be used by rootkits to enumerate minifilter drivers in the system, sometimes in preparation for installing a hook. Alternatives are the

`FltEnumerateInstances` and `FltGetFilterFromName` functions.

`FltRegisterFilter`

Registers a new minifilter driver. May be seen in rootkits that attempt to install minifilter drivers.

`GetAdaptersAddresses`

Retrieves the IP and MAC addresses of the host's network interfaces. Can be abused for VM and sandbox detection.

`GetAsyncKeyState`

Retrieves a certain keyboard key state. Can be used for sandbox evasion (for example, to wait for a certain keypress).

`GetComputerName(Ex)`

Retrieves the computer name of the system and is sometimes used to identify sandbox-related computer names.

`GetCursorPos`

Gets the current mouse cursor position. Can be used for human-interaction detection for sandbox evasion purposes.

`GetDiskFreeSpace(Ex)`

Returns the free disk space on the system and can be used to detect an analysis environment, especially if the VM is configured with a small amount of disk space.

`GetForegroundWindow`

Gets information about the active foreground window and is sometimes used for analysis tool detection or sandbox evasion.

GetKeyboardLayout

Returns the active keyboard language of the host and is sometimes used for target profiling.

GetKeyboardLayoutList

Returns a complete list of all keyboard languages that are installed on the host.

GetLastError

Retrieves the calling thread's last error value. Can be used in combination with `SetLastError` and `OutputDebugString` to detect debuggers and for other anti-analysis purposes.

GetLocalTime

Gets the current date and time of the system. Can be used to detect debugging or for other anti-analysis techniques, such as time bombing.

GetLogicalProcessorInformation(Ex)

Returns information about the system's processor. Can be used to identify a VM or sandbox environment.

GetModuleFileName(Ex)

Retrieves the path for the file that contains a specific module, or retrieves the path of the executable file of the current process. Can be used to enumerate loaded modules, such as anomalous modules injected by analysis tools, or to retrieve its own executable's path.

GetModuleHandle(Ex)

Returns a handle to a loaded module.

GetPhysicallyInstalledSystemMemory

Returns the amount of physical system memory. Can be abused to identify a VM or sandbox environment.

GetProcAddress

Gets the procedure address of a function. Can be combined with `LoadLibrary` to dynamically load libraries and functions for endpoint defense evasion and anti-analysis.

GetSystemFirmwareTable

Retrieves various firmware tables from the system. Can be used to search for hypervisor-related firmware as a VM and sandbox detection technique. An alternative is the `EnumSystemFirmwareTables` function.

GetSystemInfo

Returns information about the system. Can be used to detect a VM environment.

GetSystemMetrics

Returns information about system metrics and configurations.

GetSystemTime

See `GetLocalTime`.

GetTcpTable

Returns the system's IPv4 TCP connection table. May be used to detect a VM or sandbox that isn't connected to a network or the internet.

GetThreadContext

Retrieves the context of the current thread. Can be used for detecting hardware breakpoints. An alternative is the `Wow64GetThreadContext` function.

`GetThreadLocale`

Returns locale information about a running thread, such as the language in use. Can be used for target profiling.

`GetTickCount`

Retrieves the number of milliseconds that have elapsed since the system was started. Can be used for many anti-analysis techniques, such as debugger detection. An alternative is the `GetTickCount64` function.

`GetUserDefaultUILanguage`

Returns the currently logged-in user's interface language. Can be used for the same purposes as `GetThreadLocale`. Alternatives are the `GetSystemDefaultUILanguage`, `GetSystemDefaultLCID`, `GetUserDefaultLCID`, and `GetProcessPreferredUILanguages` functions.

`GetVersion(Ex)`

Retrieves the version information for the operating system. Can be used for target profiling.

`GetWindowText`

Obtains the title text of the window. Can be used to detect malware analysis tools.

`GlobalAddAtom(Ex)`

Adds a string to the global atom table. *See also* `AddAtom`.

GlobalGetAtomName

Retrieves the string in the specified global atom. *See also* `AddAtom`.

ImpersonateLoggedOnUser

Allows the calling thread to impersonate the security context of the logged-in user. *See also* `DuplicateToken(Ex)`.

InitiateShutdown

Shuts down and restarts the system. Can be used as an anti-analysis and anti-sandbox technique. An alternative is the `InitiateSystemShutdown(Ex)` function.

InternetConnect

Opens an FTP or HTTP internet connection. Can be used for many different evasion techniques, such as identifying whether or not a system is connected to the internet to detect a sandbox or VM. Often used in combination with `InternetOpen` and `InternetReadFile`.

IsDebuggerPresent

Checks whether the calling process is being debugged. *See also* `CheckRemoteDebuggerPresent`.

IsProcessorFeaturePresent

Returns the status of various processor features, which can indicate a VM.

LoadLibrary

Loads a module into the process address space of the calling process. *See also* `GetProcAddress`.

Module32First

Gets information about the first module loaded into a process. Can be used in combination with `Module32Next` to enumerate and identify modules associated with analysis tools.

NtCreateTransaction

Creates a new NTFS transaction object. Can be used for process manipulation techniques, such as process doppelganging.

NtLoadDriver

Loads a driver into the system. Can be invoked to load a malicious kernel module.

NtMapViewOfSection

Maps a view of a section into the process address space of a target process. Can be used to manually map a library or code into memory as part of process injection.

NtOpenDirectoryObject

Can be used to query device and driver objects on the system. Sometimes used to locate VM-related artifacts as part of VM detection. An alternative is the `NtQueryDirectoryObject` function.

NtQueryInformationProcess

Returns a great deal of information about a target process. Can be used to identify attached debuggers.

NtQueryObject

Returns information on different operating system objects. Can be used to identify debugger objects, indicating to the malware that it is being debugged.

NtQuerySystemInformation

Returns a lot of different system information. Can be used to enumerate firmware tables to identify a VM or sandbox.

NtQuerySystemTime

See `GetLocalTime`.

NtSetInformationThread

Sets the priority of a thread. Can be used to hide code execution from a debugger or, in certain circumstances, to crash the debugger.

NtUnmapViewOfSection

Unmaps a view of a section from memory. Sometimes used as part of process injection techniques.

OpenMutex

Opens a mutex object. *See also* `CreateMutex`.

OpenProcess

Opens a process object and is often a precursor to process injection. An alternative is the `NtOpenProcess` function.

OpenProcessToken

Opens the access token of a process and is often a precursor to techniques such as privilege elevation. *See also* `AdjustTokenPrivileges`.

OpenService

Opens a service. Can be used to identify services related to sandboxes and hypervisors.

OpenThread

Opens a thread object. Can be used for process injection techniques, such as thread hijacking.

OutputDebugString

Sends a string to the debugger. Can be used for anti-debugging purposes. *See also* GetLastError.

PostMessage

Closes application windows when the parameter `WS_CLOSE` is passed to the window's handle. Can be used as an anti-analysis technique.

Process32First

Gathers information about the first process in a process snapshot. *See also* CreateToolhelp32Snapshot.

Process32Next

Gathers information about the next process in a process snapshot. *See also* CreateToolhelp32Snapshot.

PsLookupProcessByProcessID

Gets a pointer to a process's `EPROCESS` structure. Sometimes used by rootkits in preparation for evasion techniques such as DKOM.

PsSetCreateProcessNotifyRoutine(Ex)

Registers a driver callback that will be triggered when any new process is created or terminated. Sometimes used by rootkits to monitor process creations.

PsSetCreateThreadNotifyRoutine(Ex)

Similar to `PsSetCreateProcessNotifyRoutine(Ex)`, but for thread creation.

PsSetLoadImageNotifyRoutine(Ex)

Registers a callback that will be triggered when a process loads an image into memory, such as a DLL module, or when a new driver is loaded. Sometimes used by rootkits to monitor module loads.

QueryPerformanceCounter

Queries the processor's performance counter and returns the current value. Can be used as part of anti-debugging and VM detection techniques.

QueueUserAPC

Queues a new asynchronous procedure call and is sometimes invoked for process injection techniques, such as APC injection. An alternative function is `NtQueueApcThread`.

ReadProcessMemory

Reads data from a memory region inside a target process. Can be used for many reasons, such as inspecting process memory for hooks as part of anti-hooking techniques. An alternative is the `NtReadVirtualMemory` function.

RegEnumKey(Ex)

Enumerates a registry key. Can be used for VM detection or to otherwise identify registry keys of interest.

RegEnumValue

Enumerates a registry key value.

RegOpenKey(Ex)

Opens a registry key for reading or writing.

ResumeThread

Resumes execution of a thread and is often used for process injection, such as process hollowing. An alternative is the `NtResumeThread` function.

RollbackTransaction

Rolls back an NTFS transaction and is used for process manipulation, such as in the process-doppelganging technique.

RtlCopyMemory

Copies the contents of a source buffer in memory to another memory region. Can be invoked to write malicious code into memory, such as injecting a hook.

RtlQueryProcessHeapInformation

Returns information about the current process's heap. Can be used to detect a debugger.

RtlZeroMemory

Fills a region of memory with zeros. Can be used as an anti-forensics and defense evasion technique.

SetFileAttributes

Sets various attributes for a file or directory. Can be used to hide files and directories via the `hidden` attribute.

SetFileTime

Sets various timestamps for a file or directory. Can be used to falsify file timestamps (timestomping).

SetPriorityClass

Sets the priority of a process. Can be abused to lower the priority of endpoint defense processes in an attempt to circumvent them.

SetThreadContext

Sets the context of a thread and is sometimes used for process injection techniques, specifically process hollowing.

SetUnhandledExceptionFilter

Allows the calling program to supersede the top-level exception handler. Can be used as part of anti-debugging and covert code execution techniques.

SetWindowsHookEx

Installs an application-defined hook. Can be used for multiple purposes, such as hooking keyboard and mouse events and injecting malicious code.

Sleep(Ex)

Suspends the execution of a thread for a specified amount of time. Can be used for many malicious purposes, such as sandbox evasion and

various anti-debugging techniques.

StartService

Starts a service on the system. Can be used to establish persistence or load a malicious module.

SuspendThread

Suspends a thread and is used in certain process injection techniques, such as thread hijacking. An alternative is the `Wow64SuspendThread` function.

TerminateProcess

Terminates a specified process. Can be used as an anti-analysis technique.

VirtualAlloc(Ex)

Allocates (reserves or commits) a region of virtual memory and is part of various process injection techniques. An alternative is the `NtAllocateVirtualMemory` function.

VirtualQuery(Ex)

Returns information about a region of memory. Can be used to detect hardware and memory breakpoints. An alternative is the `NtQueryVirtualMemory` function.

WriteProcessMemory

Writes data to a region of memory in a process and is used as part of various process injection techniques. An alternative is the `NtWriteVirtualMemory` function.

